CS6025-FR01B Assignment 1

Name: Ruturaj Sushil Ghodke
Student ID: 3775789
Date: 26 February 2026

Design Summary (practical focus)

1. **Which loops you parallelized and why?**
   For the standard threaded modes, I targeted the outermost loop (the i loop handling the matrix rows). I did this because it provides the largest, most uninterrupted "chunk" of work for each thread. If I had parallelized the inner loops, the threads would constantly have to stop, sync up, and restart, which ruins performance. In Mode 2, I used #pragma omp parallel for collapse(2) to fuse the i and j loops. This flattens the 2D grid into one massive 1D loop, which is great for ensuring all 22 of my threads have plenty of individual tasks to grab, especially when testing smaller matrix sizes.

2. **Your scoping decisions (examples of private, firstprivate, etc.)?**
   Getting scoping right was crucial to avoid silent math errors. The matrices (A, B, C) and the size (N) were scoped as shared because they are simply being read from or written to in distinct, non-overlapping locations; we definitely don't want to make private copies of massive arrays. However, the loop iterators (i, j, k) absolutely had to be private. If they were shared, threads would constantly overwrite each other's loop counters. For Mode 4 (Tasks), I used firstprivate(row). This is because a task might not execute immediately, firstprivate ensures that when the task is created, it takes a personal "snapshot" of the row number it was assigned, rather than looking at a shared loop counter that has already moved on.

3. **What you used and why for sum/max?**
   To find the sum and max values of the final matrix, I relied on OpenMP's reduction(+:sumC) and reduction(max:maxC) clauses. I chose this because doing it manually (using locks) is terrible for performance. The reduction clause acts like magic behind the scenes: it gives every thread its own temporary sumC and maxC variable to update freely. Once all threads are completely done, OpenMP safely combines all those temporary variables into the final global result.

4. **Where/why you used atomic vs critical?**
   For calculating the final checksum, I had a situation where every thread needed to update a single checksum variable. I chose #pragma omp atomic over #pragma omp critical. While both prevent race conditions, critical acts like a single-file door. It forces 21 of my threads to stand around idling while one thread enters the room to do a tiny piece of math. atomic, on the other hand, tells the CPU hardware to perform the addition in one uninterruptible instruction. It is significantly faster and less restrictive.

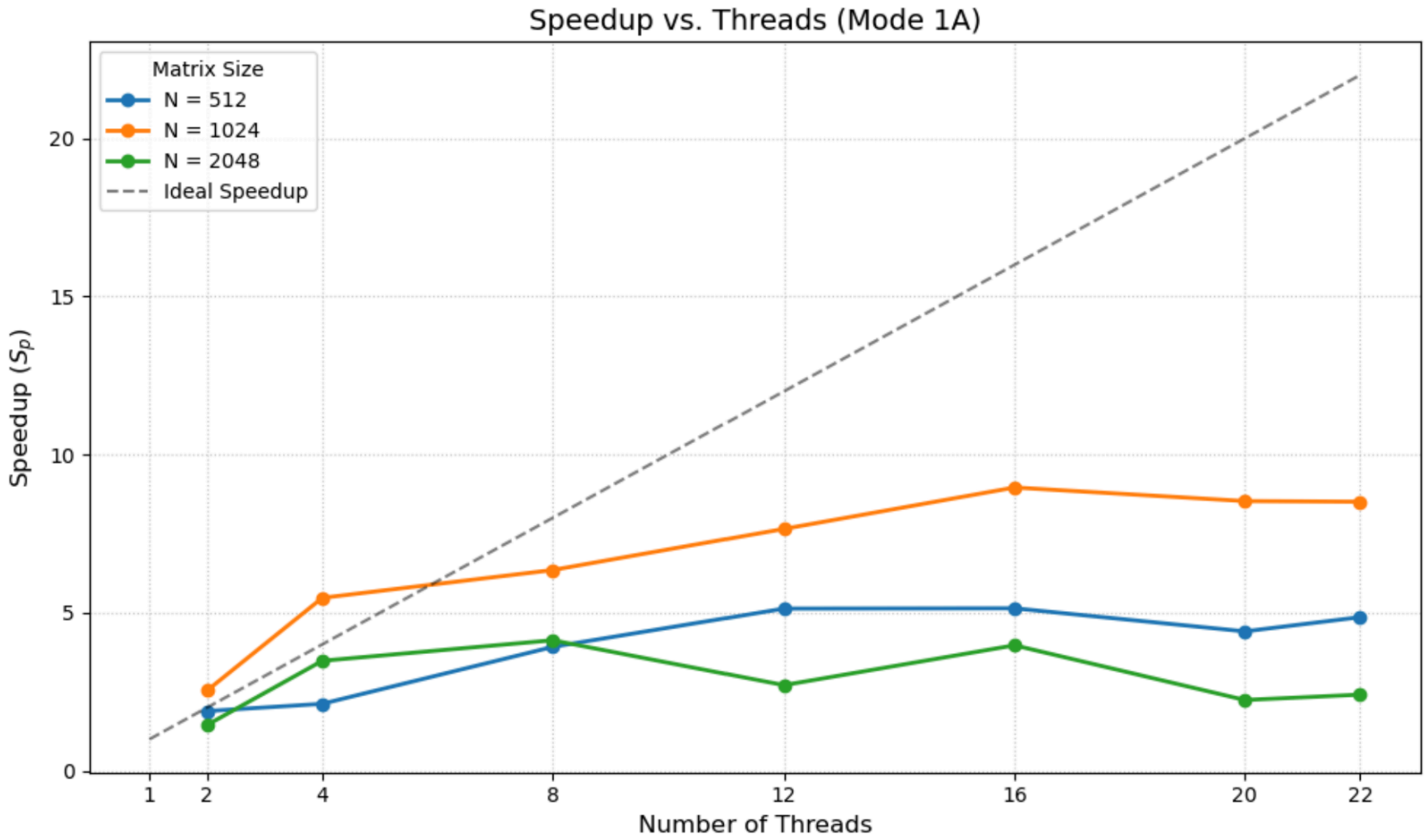
5. **Why you chose your schedule and chunk size?**
   I stuck with schedule(static) and let OpenMP handle the default chunk sizing. Matrix multiplication is a perfectly uniform problem: multiplying row 0 takes the exact same amount of time and effort as multiplying row 500. Because the workload is completely predictable, static scheduling allows the program to divide the work up evenly before the loops even start, resulting in basically zero runtime overhead.

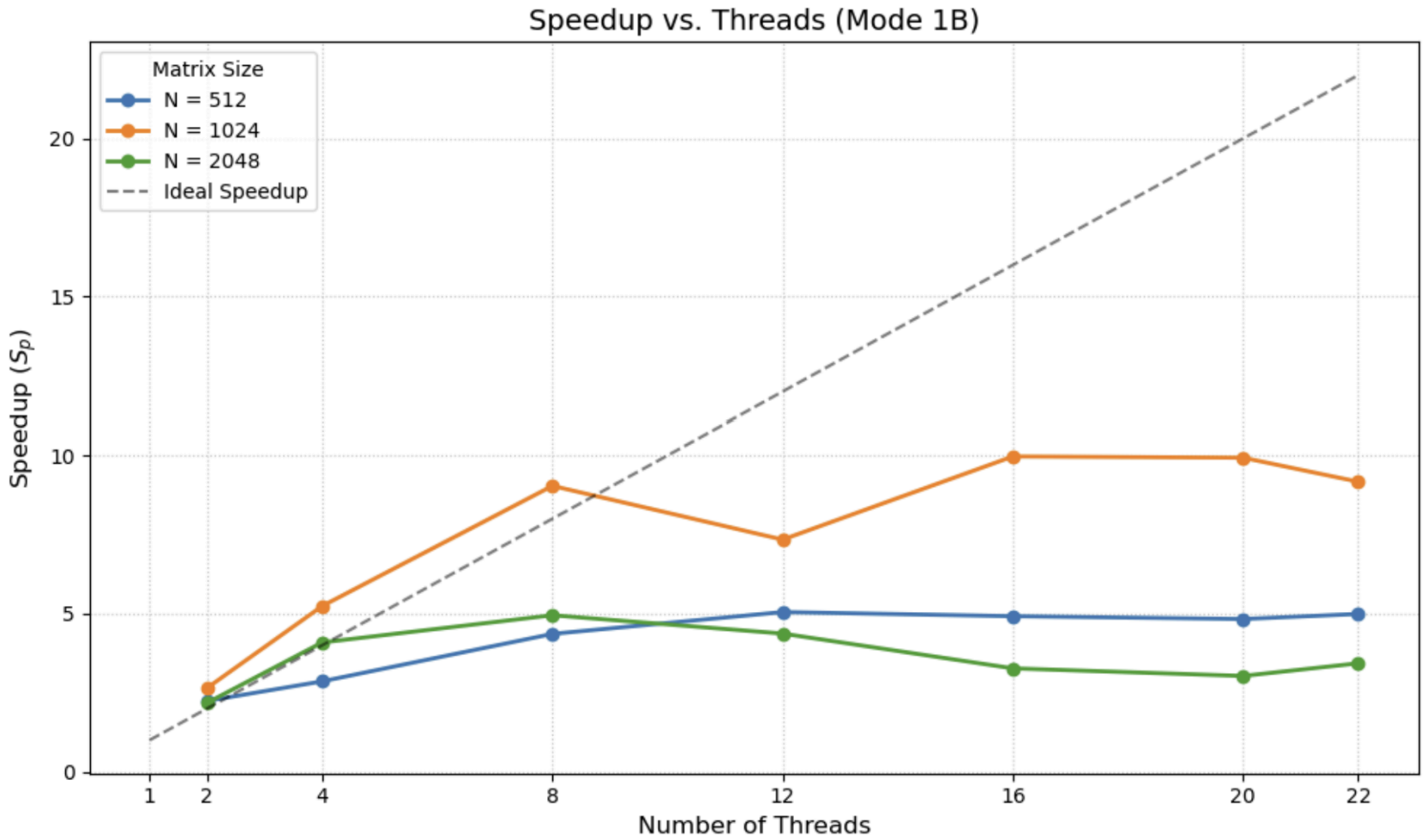
Result Table

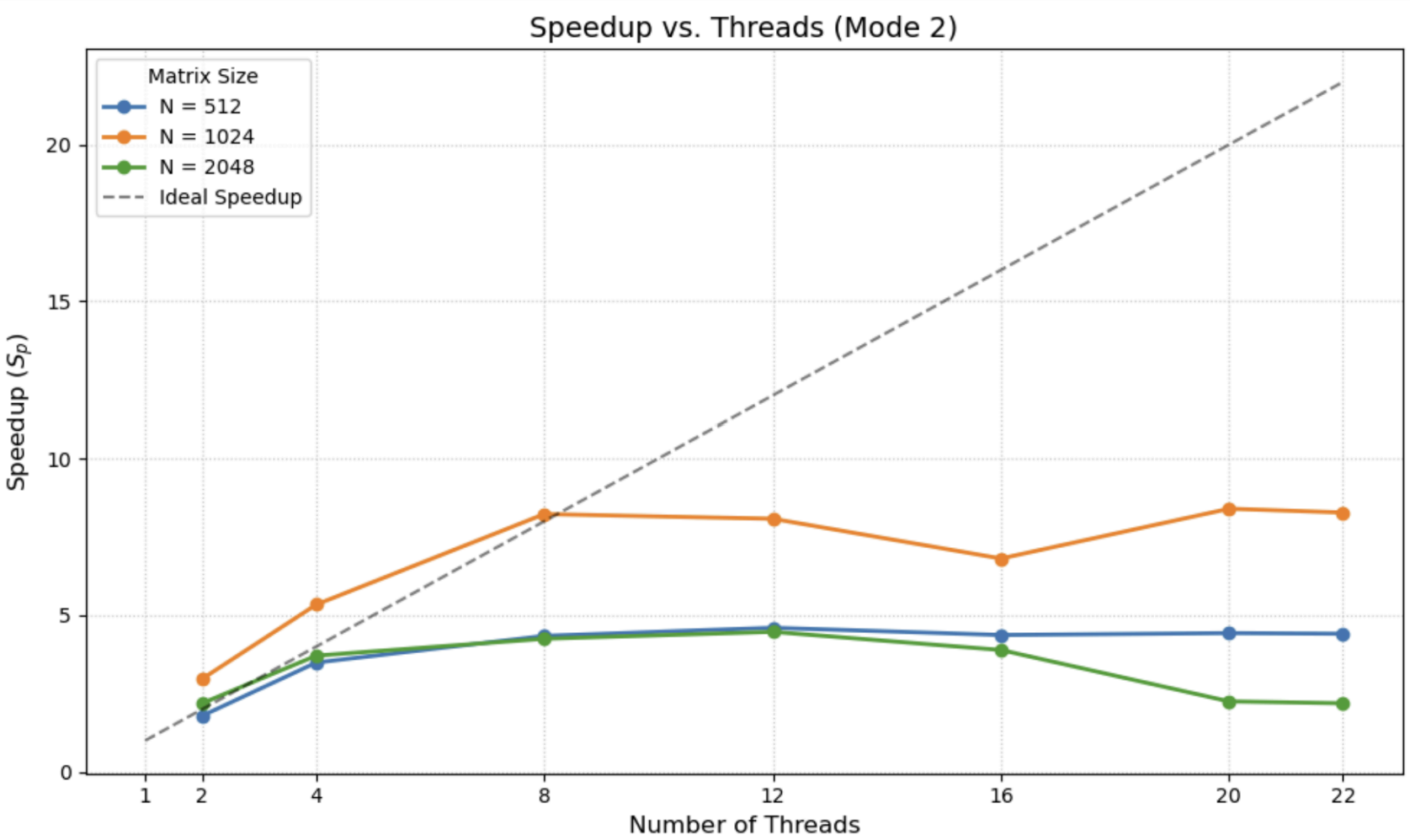
The results table is in an excel sheet called **A1_readings.xlsx**, which is provided alongside this assignment submission folder.
The excel sheet has a table of runtime averages from 10 runs for N=(512, 1024, 2048) and threads=(1, 2, 4, 8, 12, 16, 20, 22 (max)) along with speedup and efficiency values for modes 0-6.

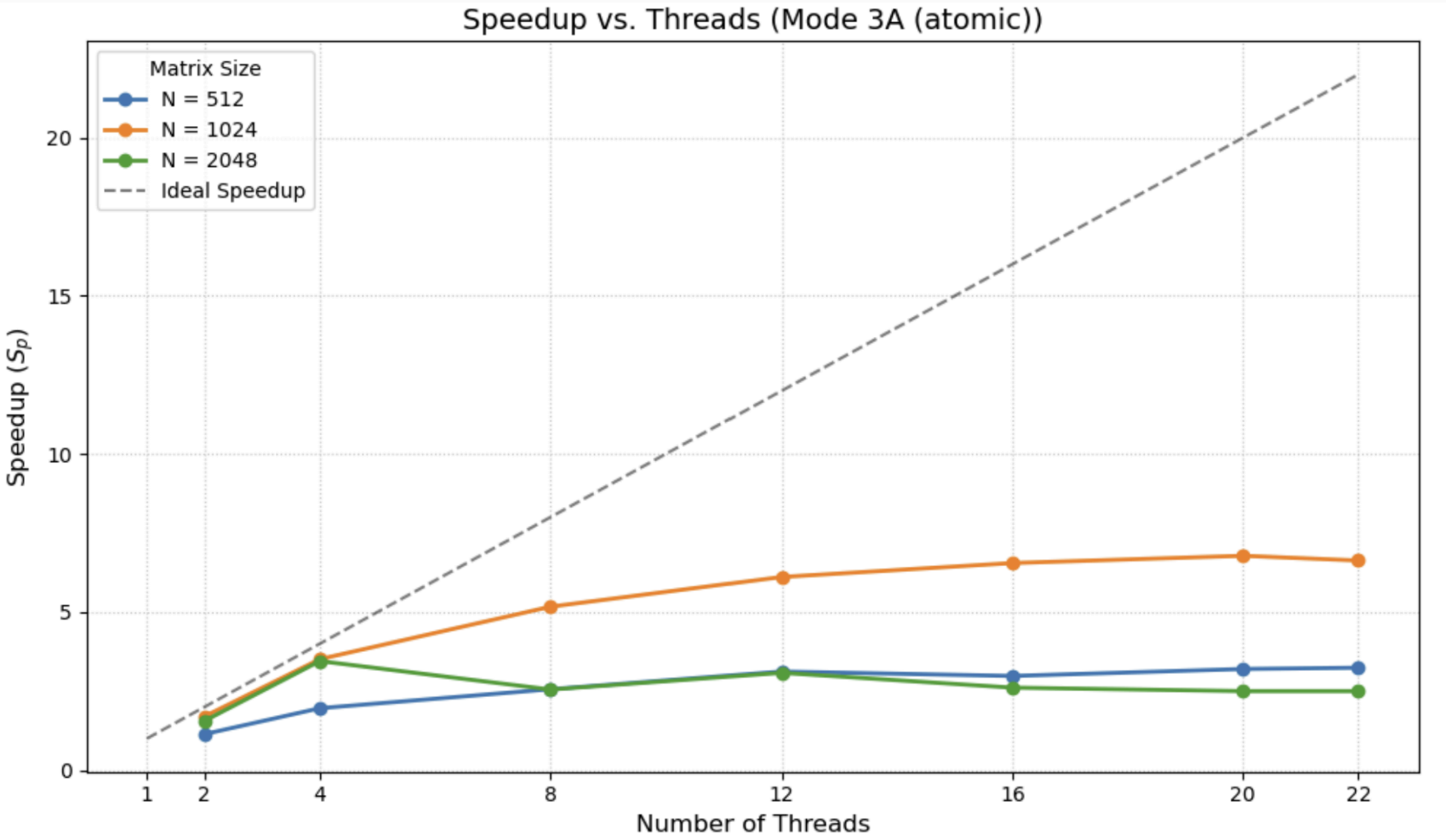
Plots

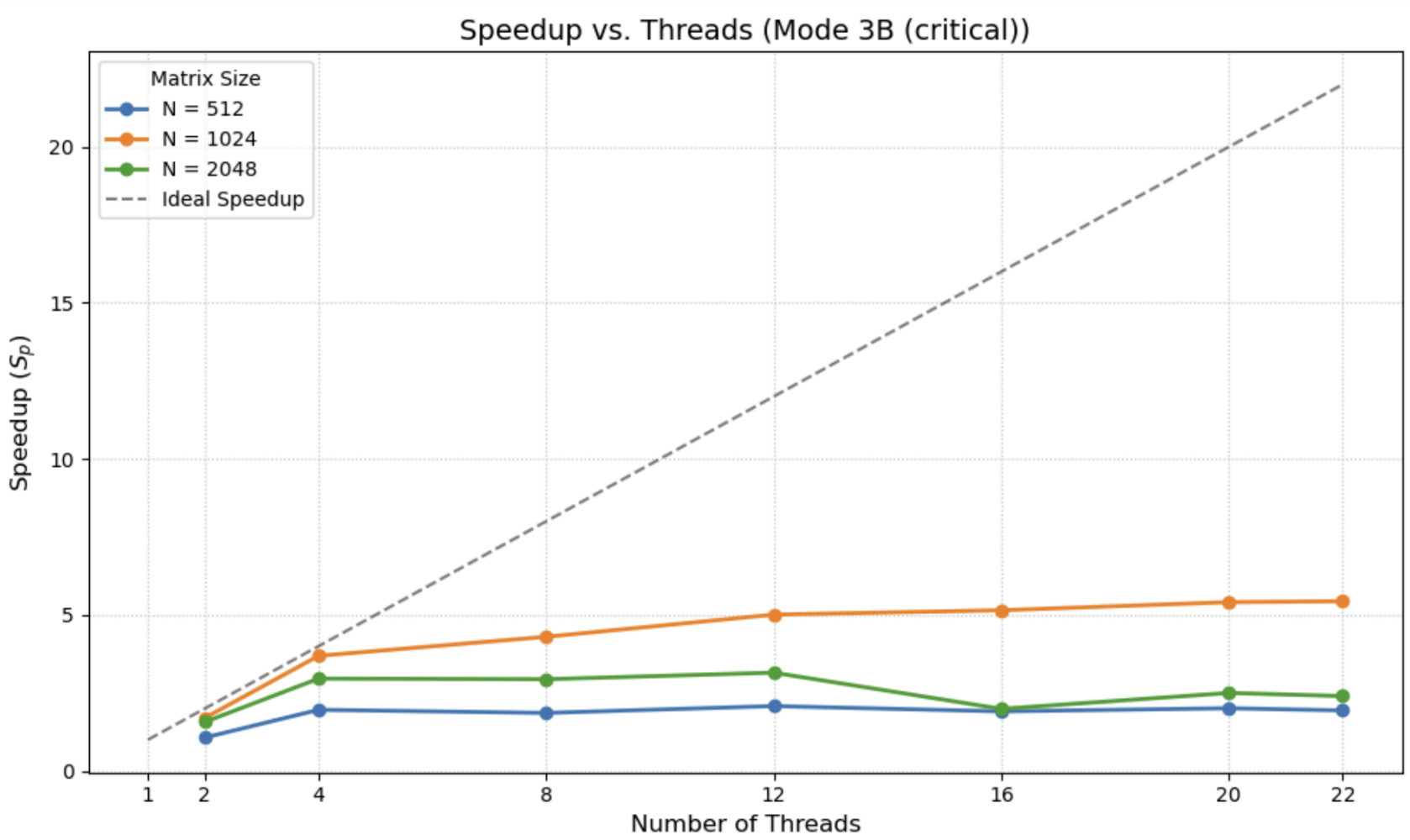
Speedup vs Threads (All Modes except 0 and 5 as both run on single threads)

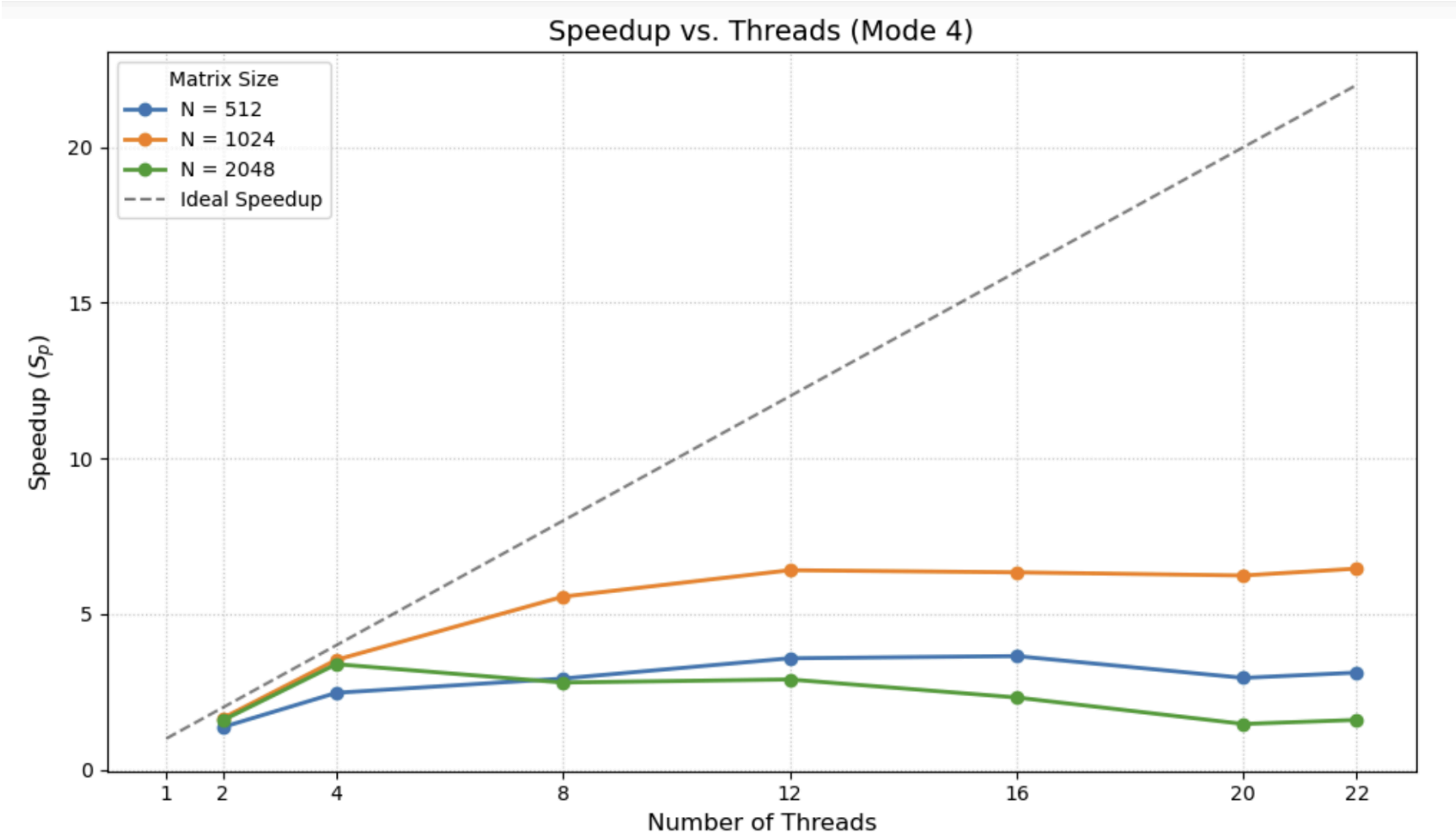


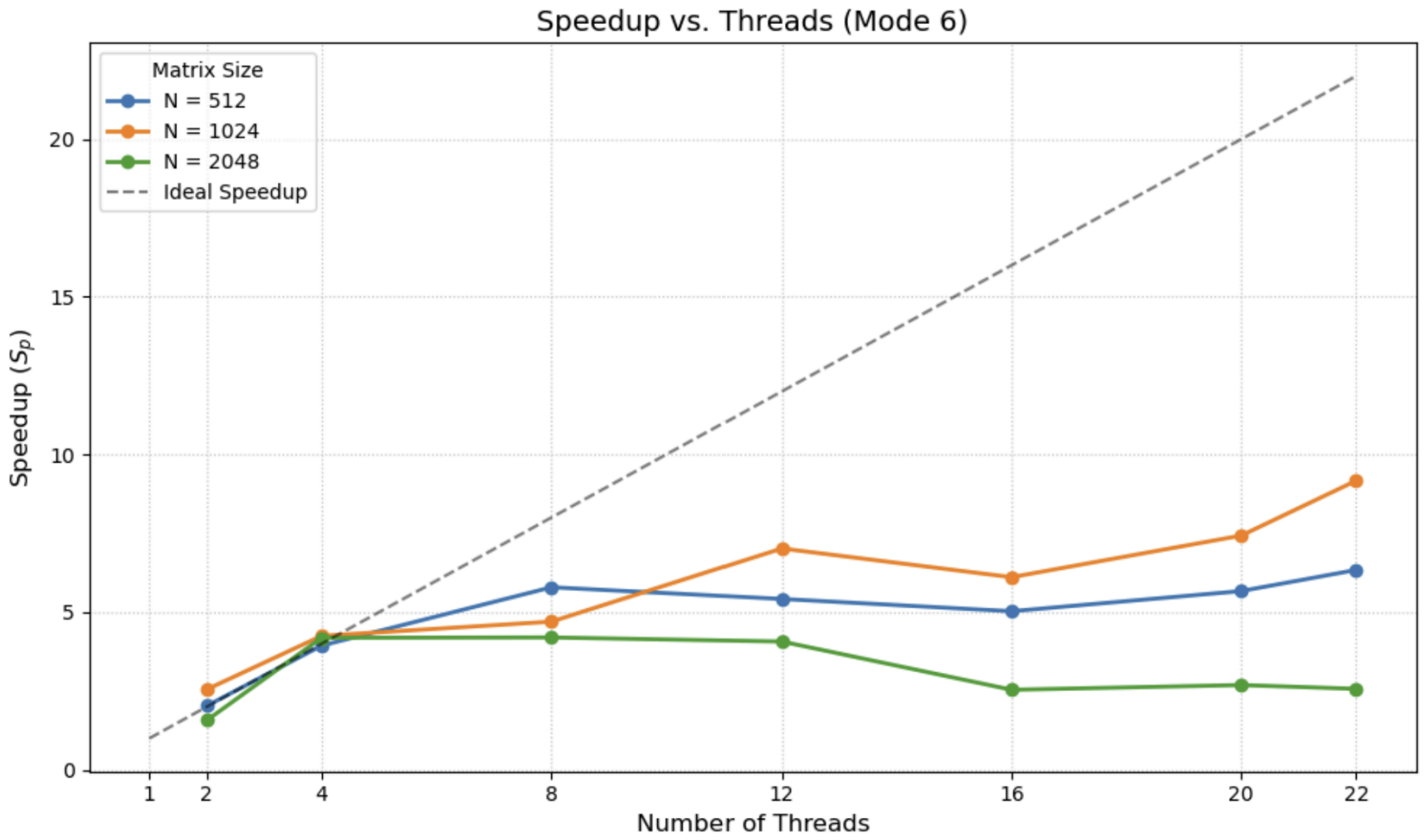

Speedup vs. Threads (Mode 1B)
Matrix Size
N = 512
N = 1024
N = 2048
Ideal Speedup
Speedup (S_p)
Number of Threads
0
5
10
15
20
1
2
4
8
12
16
20
22


Speedup vs. Threads (Mode 2)
Matrix Size
N = 512
N = 1024
N = 2048
Ideal Speedup
Speedup (S_p)
0
5
10
15
20
1
2
4
8
12
16
20
22
Number of Threads

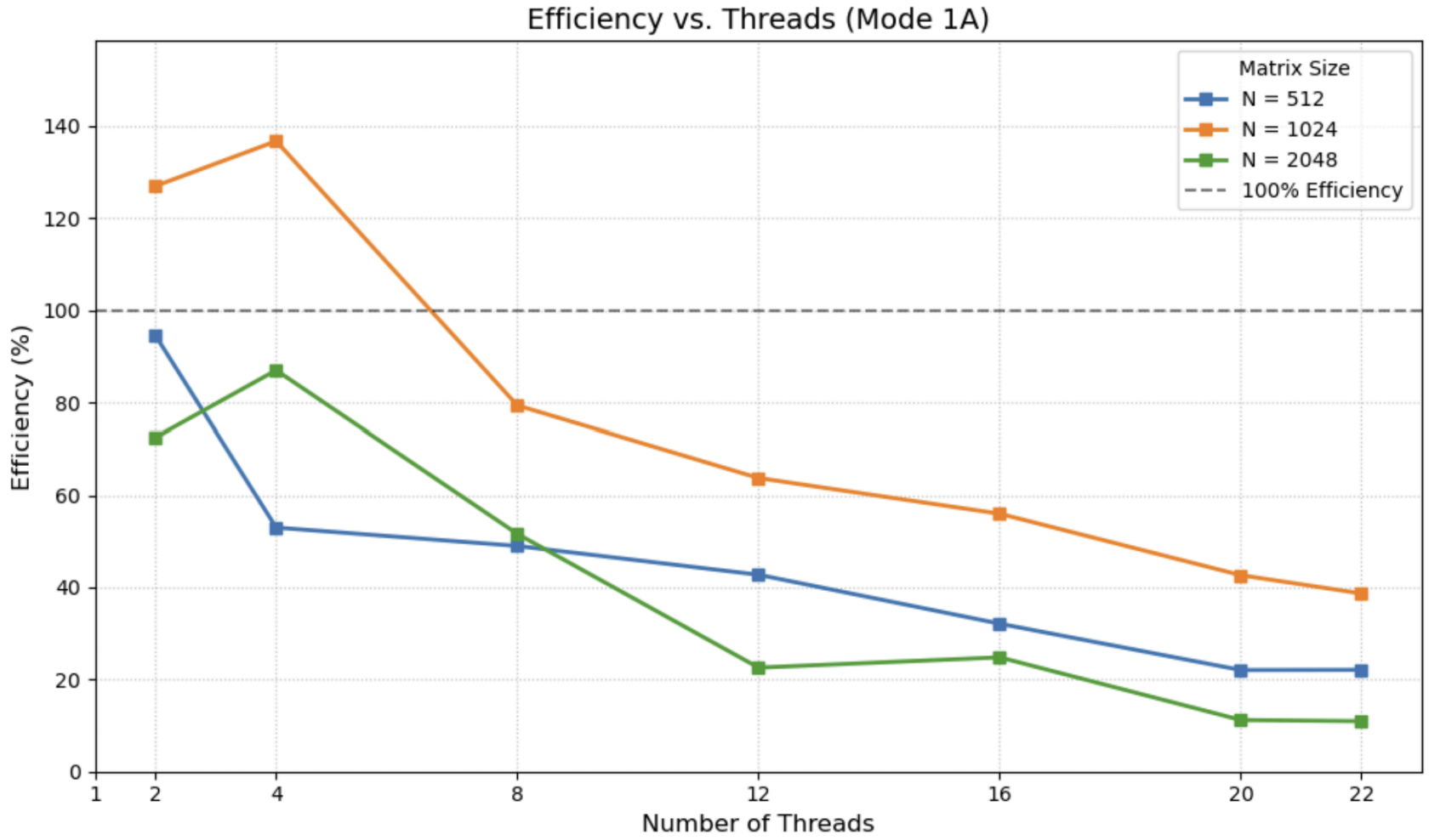

Speedup vs. Threads (Mode 3A (atomic))
Matrix Size
N = 512
N = 1024
N = 2048
Ideal Speedup
Speedup (S_p)
0
5
10
15
20
1
2
4
8
12
16
20
22
Number of Threads

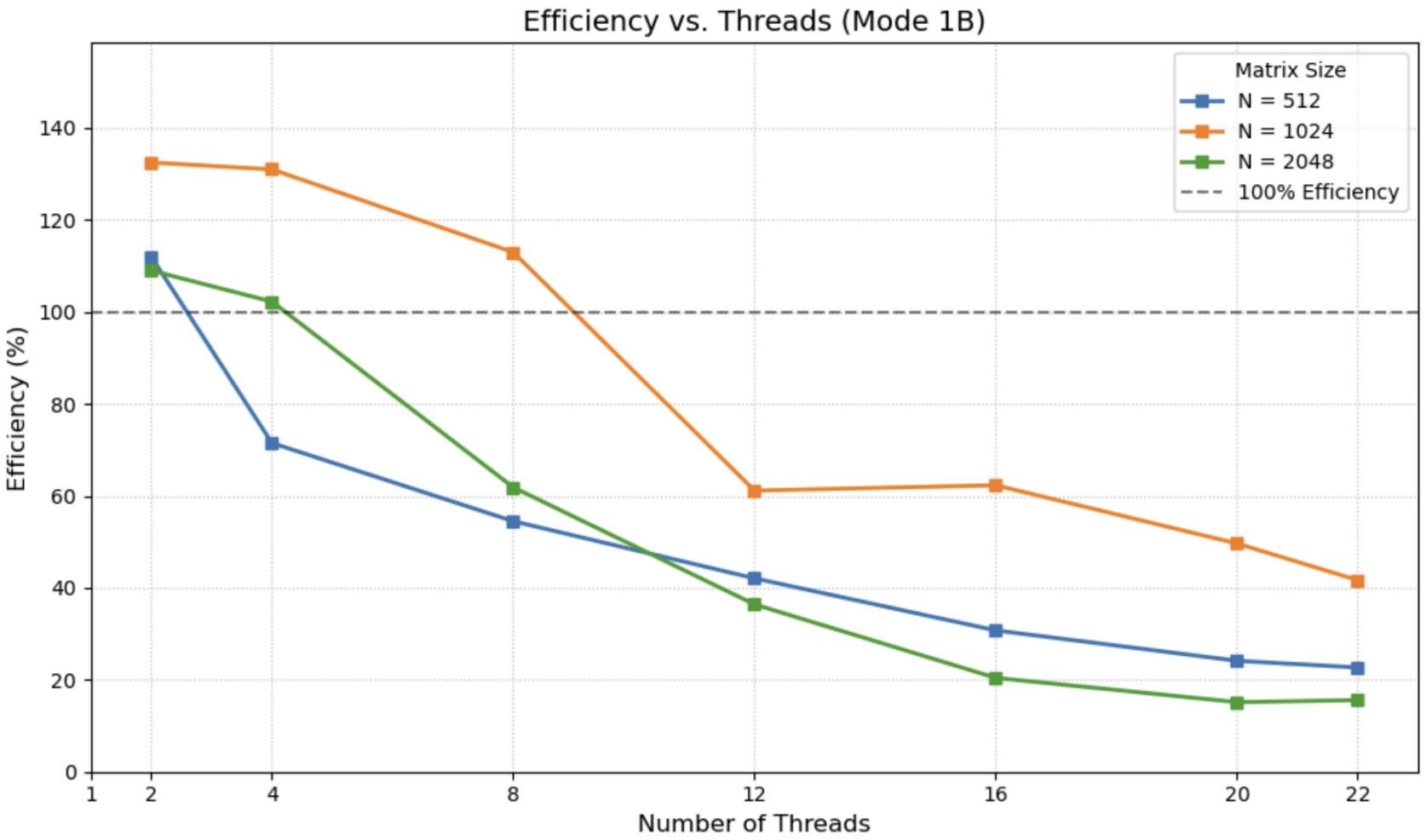

Speedup vs. Threads (Mode 3B (critical))
Matrix Size
N = 512
N = 1024
N = 2048
Ideal Speedup
Speedup (S_p)
0
5
10
15
20
1
2
4
8
12
16
20
22
Number of Threads

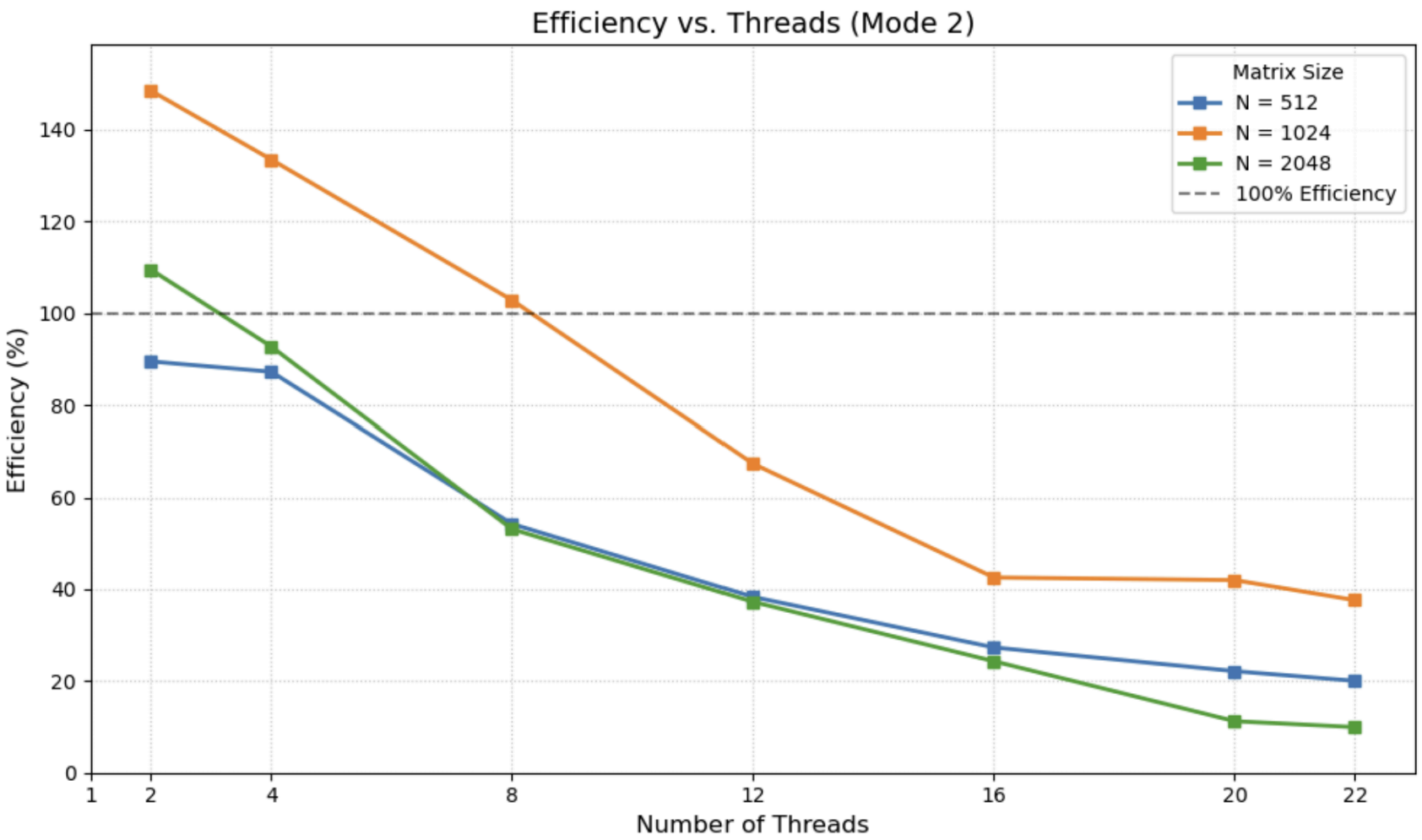

Speedup vs. Threads (Mode 4)
Matrix Size
N = 512
N = 1024
N = 2048
Ideal Speedup
Speedup (S_p)
0
5
10
15
20
1
2
4
8
12
16
20
22
Number of Threads

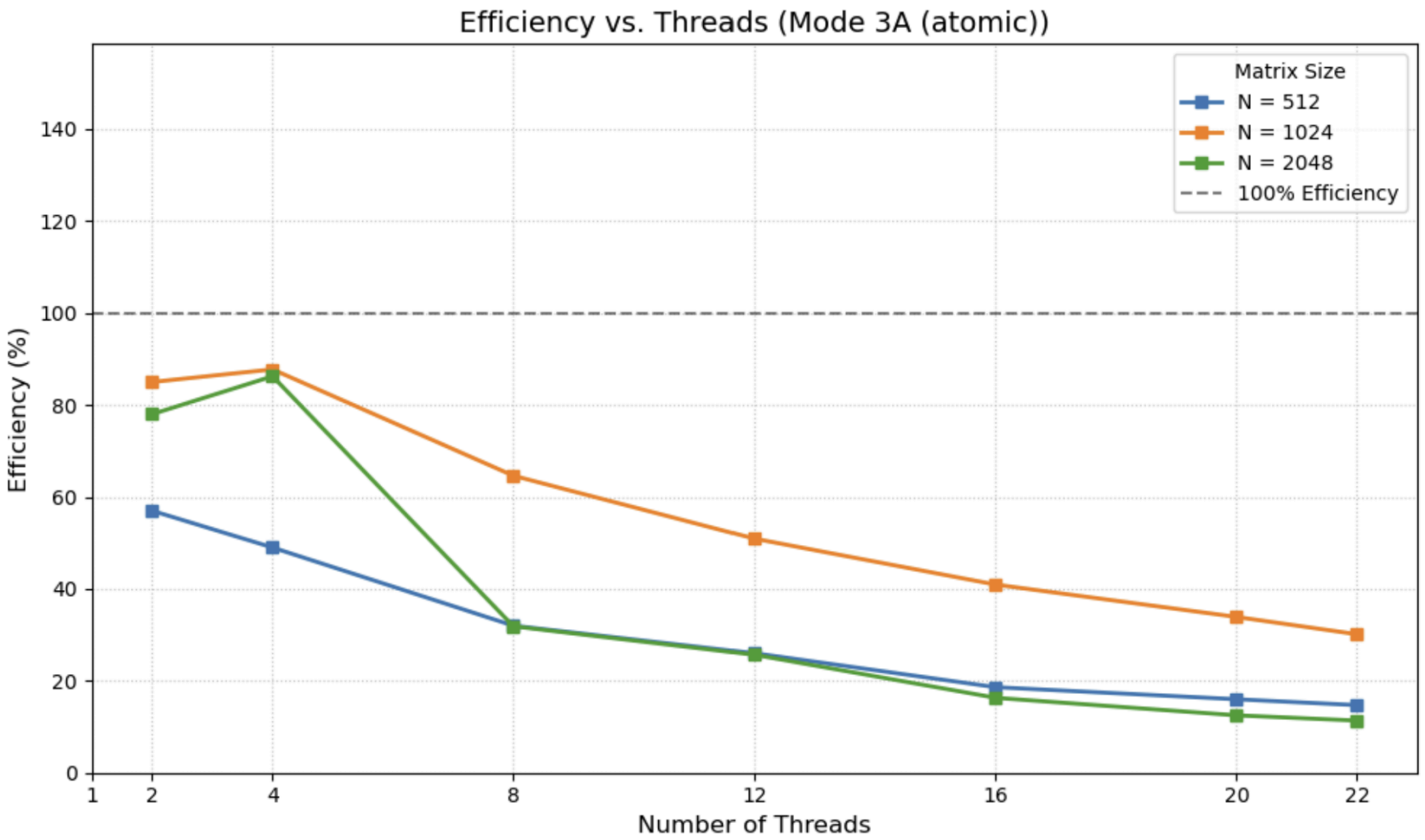

Speedup vs. Threads (Mode 6)
Matrix Size
N = 512
N = 1024
N = 2048
Ideal Speedup
Speedup (S_p)
0
5
10
15
20
1
2
4
8
12
16
20
22
Number of Threads

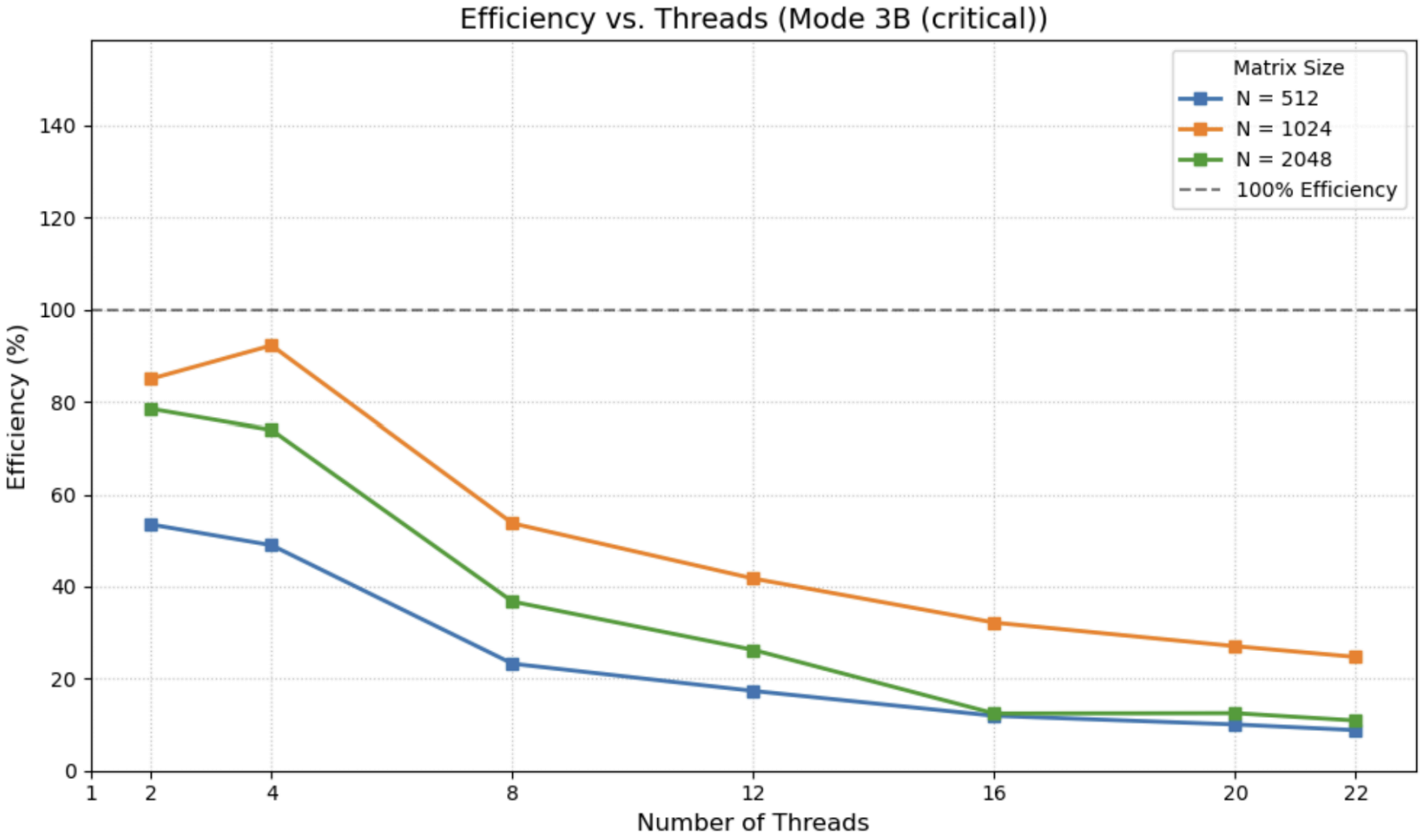
Efficiency vs Threads (All Modes except 0 and 5 as both run on single threads)

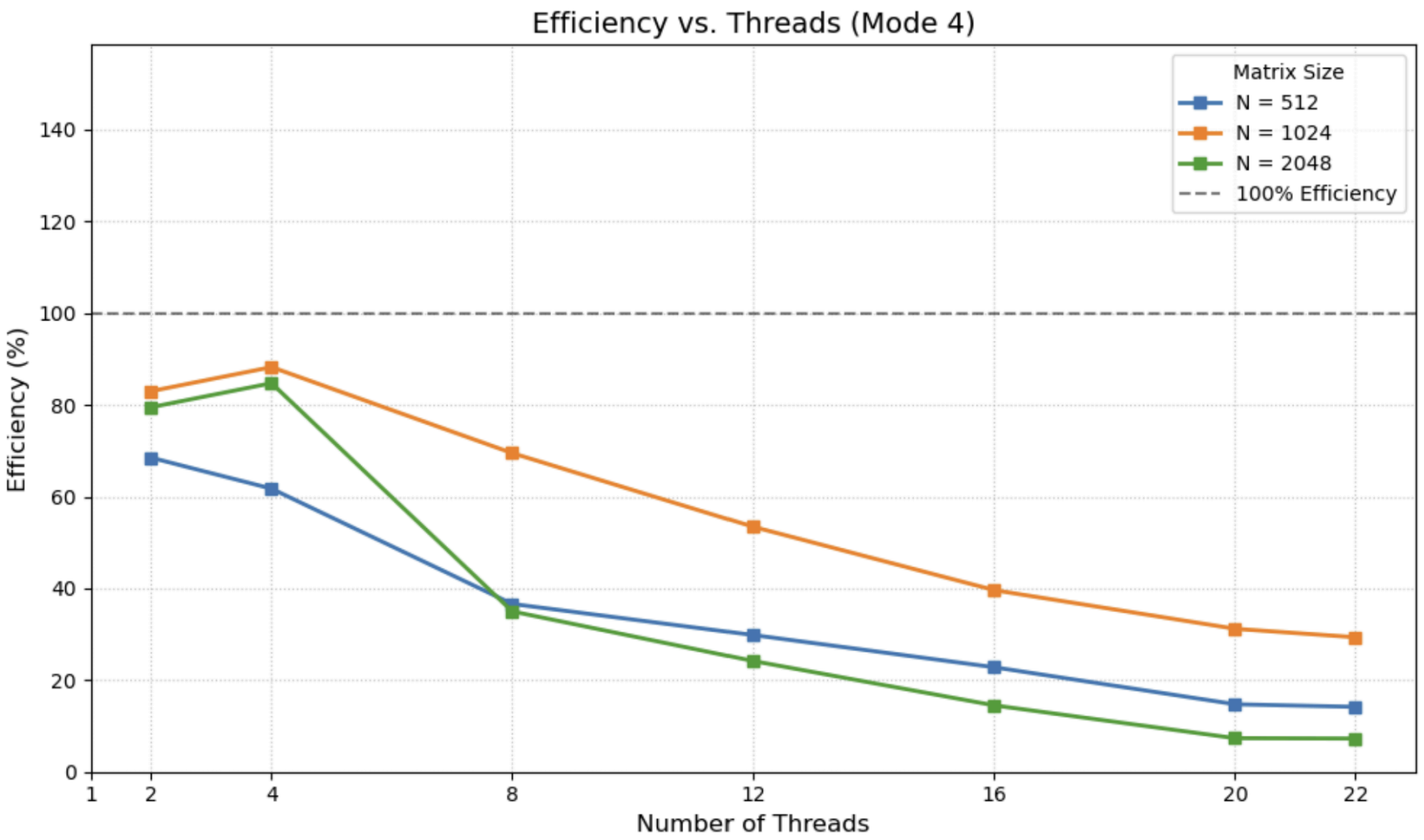


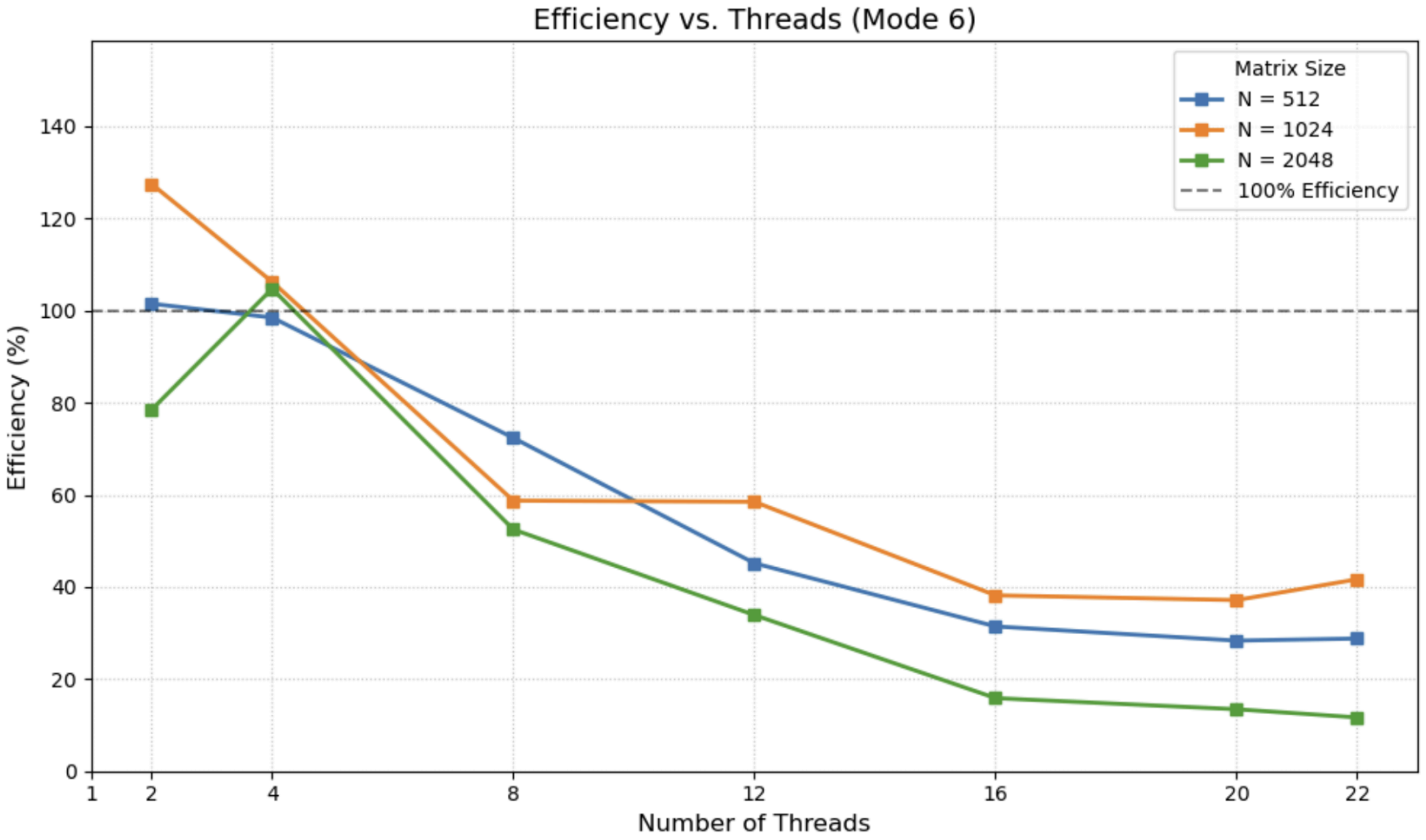

Efficiency vs. Threads (Mode 1B)
Matrix Size
N = 512
N = 1024
N = 2048
100% Efficiency
Efficiency (%)
0
20
40
60
80
100
120
140
1
2
4
8
12
16
20
22
Number of Threads


Efficiency vs. Threads (Mode 2)
Matrix Size
N = 512
N = 1024
N = 2048
100% Efficiency
Efficiency (%)
0
20
40
60
80
100
120
140
1
2
4
8
12
16
20
22
Number of Threads


Efficiency vs. Threads (Mode 3A (atomic))
Matrix Size
N = 512
N = 1024
N = 2048
100% Efficiency
Efficiency (%)
0
20
40
60
80
100
120
140
1
2
4
8
12
16
20
22
Number of Threads


Efficiency vs. Threads (Mode 3B (critical))
Matrix Size
N = 512
N = 1024
N = 2048
100% Efficiency
Efficiency (%)
0
20
40
60
80
100
120
140
1
2
4
8
12
16
20
22
Number of Threads


Efficiency vs. Threads (Mode 4)
Matrix Size
N = 512
N = 1024
N = 2048
100% Efficiency
Efficiency (%)
0
20
40
60
80
100
120
140
1
2
4
8
12
16
20
22
Number of Threads


Efficiency vs. Threads (Mode 6)
Matrix Size
N = 512
N = 1024
N = 2048
100% Efficiency
Efficiency (%)
0
20
40
60
80
100
120
140
1
2
4
8
12
16
20
22
Number of Threads

Discussion

1. **Static vs dynamic scheduling: which was better and why?**
   Static scheduling was definitely the better choice for this specific problem. Dynamic scheduling works by having a "manager" hand out small chunks of work on the fly, which is great if some loops take unexpectedly longer than others. However, in our dense matrix multiplication, every single row is identical in terms of computational workload. Because there was no imbalance to fix, the dynamic schedule's "manager" just added unnecessary communication overhead, slowing down the total execution time compared to the perfectly pre-planned static schedule.

2. **Why does efficiency drop as thread count increases?**
   The data clearly shows that efficiency plummets as we scale up to 22 threads, which is a classic demonstration of Amdahl's Law and parallel overhead. First, managing threads isn't free - spinning them up, scheduling them, and joining them takes time. When *N=512*, the actual math happens so fast that the thread-management overhead takes up a huge percentage of the total runtime. Second, as we add more threads, they all try to pull data from the RAM at the same time. This leads to cache contention and memory bottlenecks; the CPU cores end up starving while waiting for data to arrive from the system memory.

3. **Atomic vs critical: which was faster for checksum, and why?**
   As expected, atomic was noticeably faster than critical, and the gap widened as the problem size grew. #pragma omp critical is a software-level lock. It essentially pauses the parallel nature of the program, forcing a severe bottleneck where threads wait in a queue. #pragma omp atomic bypasses this by leveraging specific hardware-level instructions (like fetch-and-add) to update the memory address safely without locking down a whole block of code. For a simple addition operation like our checksum, atomic is the undisputed winner.

4. **Tasks vs loops: when did tasks help/hurt?**
   Tasks (Mode 4) generally hurt performance compared to the traditional parallel for loops (Mode 1). Tasks are an incredibly powerful tool for irregular workloads like traversing a linked list or navigating a tree where you don't know

how much work exists beforehand. But for a highly structured, perfectly predictable 2D array, tasks are overkill. The OpenMP runtime had to spend extra cycles packaging, queuing, and assigning thousands of individual row tasks, whereas a standard loop simply divides the index boundaries mathematically with almost no overhead.

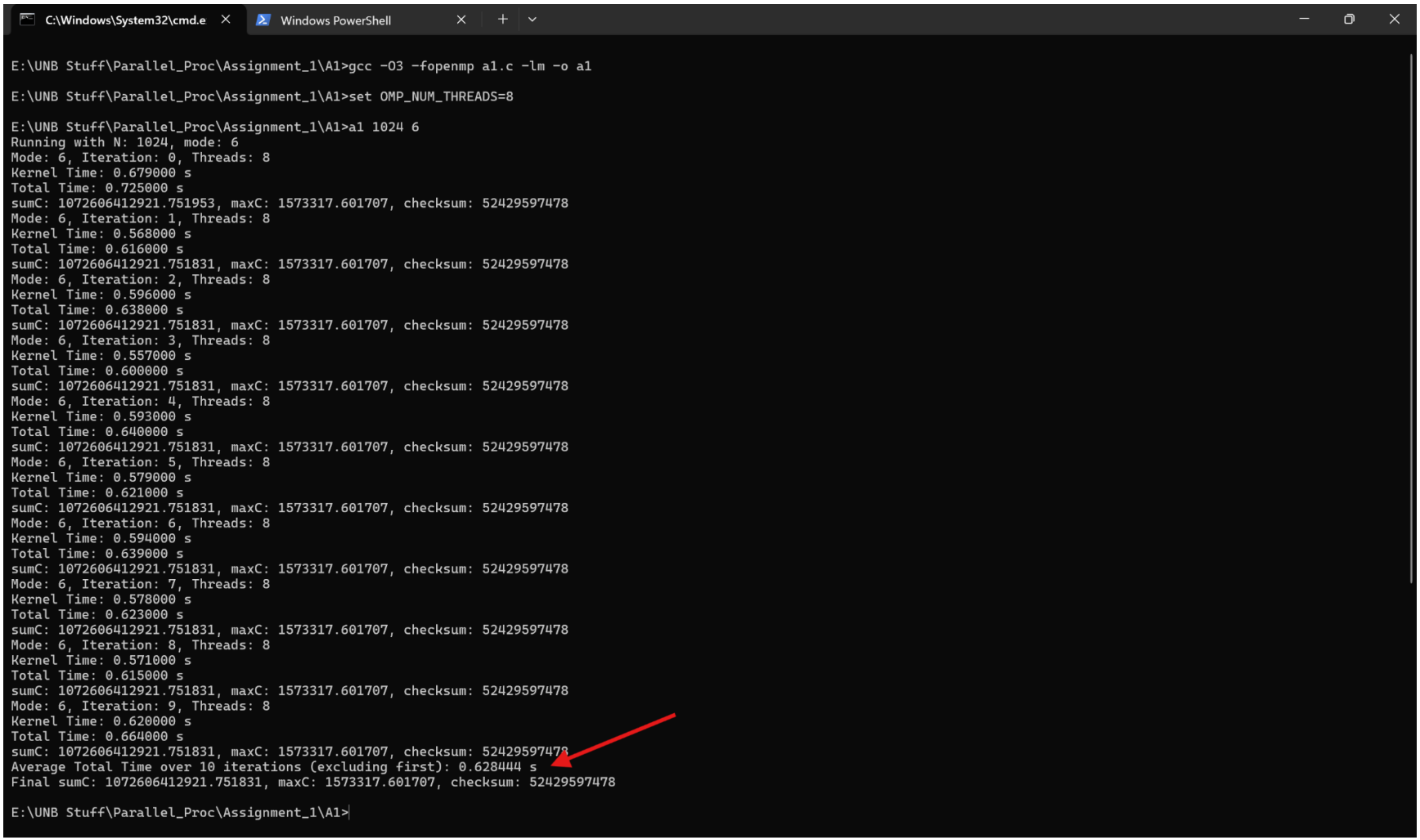
5. **SIMD: which part benefited most, and did SIMD reduce the need for many threads?**
   The innermost loop benefited the most from SIMD, as the simd reduction allowed the CPU to use its vector registers to process multiple floating-point multiplications simultaneously. Interestingly, the data for Mode 5 (SIMD with 1 thread) shows it can actually outperform several standard threads. At *N=1024*, Mode 5 is faster than Mode 0 and shows that by optimizing the data path within a single core, you can achieve speedups that rival multi-threaded execution. This proves that data-level parallelism (SIMD) is a vital first step before even trying to scale up the thread count.

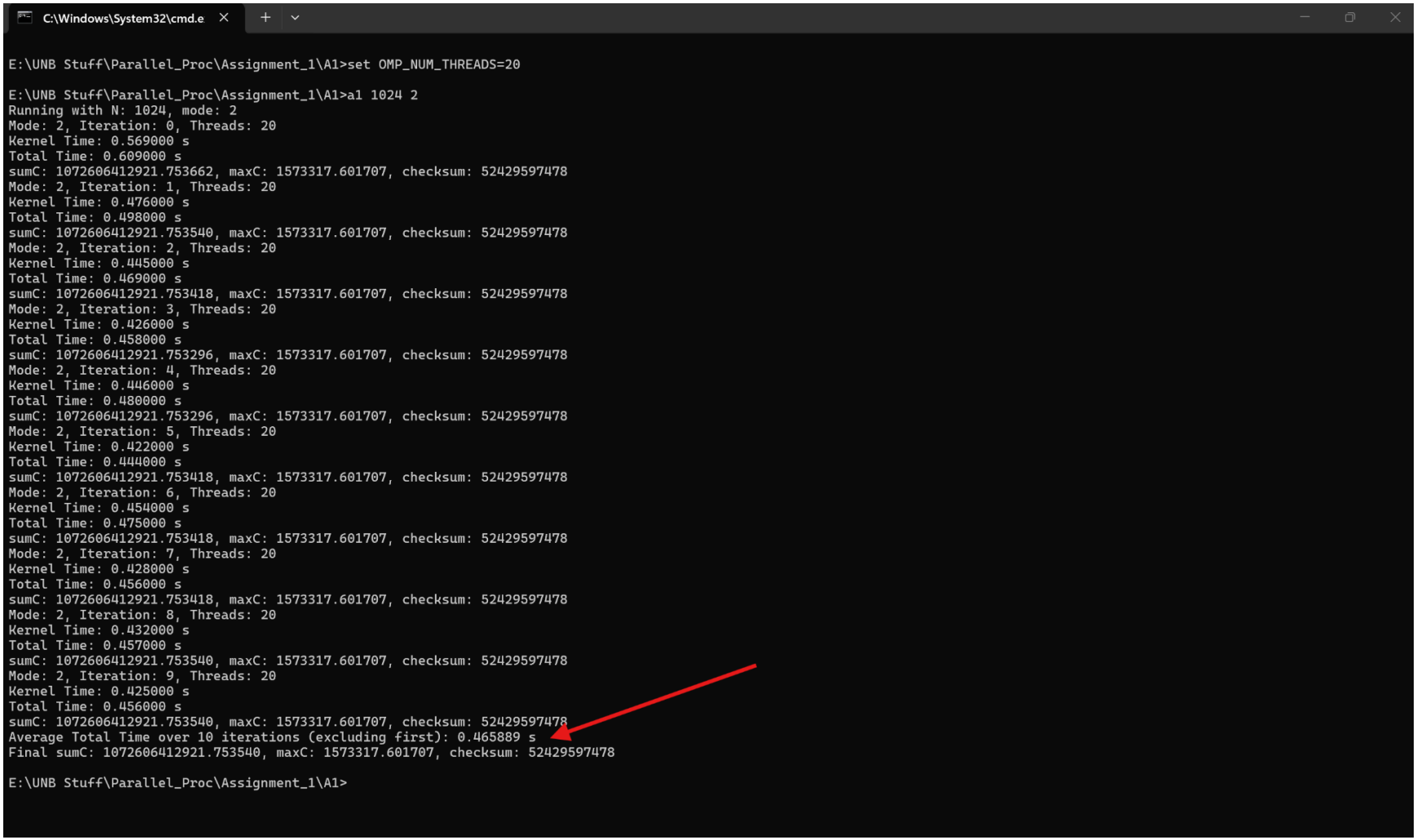
Appendix

Github Link for code

https://github.com/shotcaller/parallel-proc-a1

Screenshots of runtime evidence


C:\Windows\System32\cmd.e
Windows PowerShell
E:\UNB Stuff\Parallel_Proc\Assignment_1\A1>gcc -O3 -fopenmp a1.c -lm -o a1
E:\UNB Stuff\Parallel_Proc\Assignment_1\A1>set OMP_NUM_THREADS=8
E:\UNB Stuff\Parallel_Proc\Assignment_1\A1>a1 1024 6
Running with N: 1024, mode: 6
Mode: 6, Iteration: 0, Threads: 8
Kernel Time: 0.679000 s
Total Time: 0.725000 s
sumC: 1072606412921.751953, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 1, Threads: 8
Kernel Time: 0.568000 s
Total Time: 0.616000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 2, Threads: 8
Kernel Time: 0.596000 s
Total Time: 0.638000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 3, Threads: 8
Kernel Time: 0.557000 s
Total Time: 0.600000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 4, Threads: 8
Kernel Time: 0.593000 s
Total Time: 0.640000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 5, Threads: 8
Kernel Time: 0.579000 s
Total Time: 0.621000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 6, Threads: 8
Kernel Time: 0.594000 s
Total Time: 0.639000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 7, Threads: 8
Kernel Time: 0.578000 s
Total Time: 0.623000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 8, Threads: 8
Kernel Time: 0.571000 s
Total Time: 0.615000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Mode: 6, Iteration: 9, Threads: 8
Kernel Time: 0.620000 s
Total Time: 0.664000 s
sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
Average Total Time over 10 iterations (excluding first): 0.628444 s
Final sumC: 1072606412921.751831, maxC: 1573317.601707, checksum: 52429597478
E:\UNB Stuff\Parallel_Proc\Assignment_1\A1>


C:\Windows\System32\cmd.e
E:\UNB Stuff\Parallel_Proc\Assignment_1\A1>set OMP_NUM_THREADS=20
E:\UNB Stuff\Parallel_Proc\Assignment_1\A1>a1 1024 2
Running with N: 1024, mode: 2
Mode: 2, Iteration: 0, Threads: 20
Kernel Time: 0.569000 s
Total Time: 0.609000 s
sumC: 1072606412921.753662, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 1, Threads: 20
Kernel Time: 0.476000 s
Total Time: 0.498000 s
sumC: 1072606412921.753540, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 2, Threads: 20
Kernel Time: 0.445000 s
Total Time: 0.469000 s
sumC: 1072606412921.753418, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 3, Threads: 20
Kernel Time: 0.426000 s
Total Time: 0.458000 s
sumC: 1072606412921.753296, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 4, Threads: 20
Kernel Time: 0.446000 s
Total Time: 0.480000 s
sumC: 1072606412921.753296, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 5, Threads: 20
Kernel Time: 0.422000 s
Total Time: 0.444000 s
sumC: 1072606412921.753418, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 6, Threads: 20
Kernel Time: 0.454000 s
Total Time: 0.475000 s
sumC: 1072606412921.753418, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 7, Threads: 20
Kernel Time: 0.428000 s
Total Time: 0.456000 s
sumC: 1072606412921.753418, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 8, Threads: 20
Kernel Time: 0.432000 s
Total Time: 0.457000 s
sumC: 1072606412921.753540, maxC: 1573317.601707, checksum: 52429597478
Mode: 2, Iteration: 9, Threads: 20
Kernel Time: 0.425000 s
Total Time: 0.456000 s
sumC: 1072606412921.753540, maxC: 1573317.601707, checksum: 52429597478
Average Total Time over 10 iterations (excluding first): 0.465889 s
Final sumC: 1072606412921.753540, maxC: 1573317.601707, checksum: 52429597478
E:\UNB Stuff\Parallel_Proc\Assignment_1\A1>